

SOFTWARE REQUIREMENTS SPECIFICATION

(SRS)

AI Knowledge Assistant

Multi-Tenant RAG Platform

Table of Contents

1. Project Overview
2. Problem Statement
3. Objectives
4. Project Scope
 - 4.1 In Scope (This Development Cycle)
 - 4.2 Out of Scope (This Development Cycle)
5. System Actors
6. Functional Requirements
 - 6.1 Authentication & Account Management
 - 6.2 Organization Mode
 - 6.3 Personal Mode
 - 6.4 RAG / Chat Functionality
 - 6.5 Chat History
 - 6.6 Cross-Platform Client
7. Non-Functional Requirements
8. User Stories
9. Use Cases
10. Complete End-to-End Workflow
11. Edge Cases
12. Assumptions
13. Constraints
14. Success Criteria
15. Risks
16. Future Scope

1. Project Overview

The AI Knowledge Assistant is a cross-platform application that allows both organizations and individual users to interact with their own documents through natural language conversation, powered by Retrieval-Augmented Generation (RAG). Organizations get an isolated workspace where an Admin manages a shared knowledge base that all organization members can query, while individual users get a private personal workspace for their own documents. In both cases, the system answers strictly from the provided documents and explicitly states when it cannot find relevant information, avoiding fabricated ("hallucinated") answers.

2. Problem Statement

Organizations and individuals accumulate large volumes of documents (policies, manuals, notes, reports) that are difficult to search efficiently using traditional keyword search, and impossible for general-purpose AI chatbots to answer questions about, since those chatbots have no access to private or internal content. Existing "Chat with PDF" tools are typically single-user and do not address organizational needs such as shared, access-controlled knowledge bases with proper data isolation between different organizations and individuals. There is a need for a system that combines conversational AI with strict, verifiable grounding in an organization's or individual's own documents, with confidentiality boundaries fully respected.

3. Objectives

ID	Objective Description
O1	Enable organizations to create isolated, shared knowledge bases queryable by their members.
O2	Enable individual users to privately query their own documents.
O3	Guarantee strict data isolation between organizations and between personal and organizational data.
O4	Ground all AI answers strictly in retrieved document content, with an explicit "not found" fallback.
O5	Deliver a single, cross-platform client codebase (Flutter) rather than separate applications per platform.
O6	Design the system with modular, swappable AI components (LLM provider, embeddings) for flexibility and cost control.
O7	Apply production-quality software engineering practices achievable within a two-month, two-developer constraint.

4. Project Scope

4.1 In Scope (This Development Cycle)

- User authentication (registration/login) with JWT-based sessions.
- Two operational modes: Organization Mode and Personal Mode.
- Organization creation with a single Admin role per organization.
- Two roles: Admin (upload/delete organization documents) and Member (query only).
- Document upload (PDF, DOCX, TXT) and deletion, with automatic text extraction, chunking, embedding, and vector storage.
- RAG-based question answering, strictly grounded in retrieved content, with a "not found" fallback.
- Strict multi-tenant data isolation (organization-to-organization, personal-to-organization).
- Session-based chat history, extended to persistent history if time allows.
- Cross-platform Flutter client for Web and Android (iOS/Windows treated as stretch goals).
- Swappable LLM provider architecture (OpenAI or Anthropic).
- Relational database for structured data (users, organizations, document metadata) alongside ChromaDB for vector data.

4.2 Out of Scope (This Development Cycle)

The following items are deliberately excluded from the current development cycle to keep the project achievable within the two-month timeline. They are retained as documented future enhancements (see Section 16).

- Fine-grained roles beyond Admin/Member.
- Anonymous or unauthenticated public access.
- In-place document content editing/versioning.
- Third-party single sign-on (SSO) login.
- Admin analytics dashboards and audit logging.
- OCR for scanned/image-based PDFs.
- Enterprise-scale infrastructure (load balancing, horizontal scaling, CI/CD pipelines).
- iOS and Windows builds (attempted only if the timeline allows).

5. System Actors

Actor	Type	Description
Organization Admin	Human	Creates and manages the organization's knowledge base; uploads and deletes documents; may also ask questions.
Organization Member	Human	Belongs to an organization; can ask questions against the organization's knowledge base only; cannot upload or delete documents.
Personal User	Human	Independent account; uploads and queries only their own private documents.
LLM Provider	External System	OpenAI or Anthropic API; generates natural language answers from provided context.
Vector Database (ChromaDB)	External System	Stores and retrieves document embeddings for semantic search.
Relational Database	External System	Stores structured data: users, organizations, roles, document metadata, and chat history.

Note on roles: a single human user account has exactly one context at a time — the user operates either within an organization (as Admin or Member) or within their own Personal Mode. A user may hold a personal account while also belonging to an organization; however, their personal documents and organizational documents remain fully isolated data sets and are never mixed during retrieval.

6. Functional Requirements

6.1 Authentication & Account Management

ID	Requirement
FR-1	The system shall allow users to register with an email address and password.
FR-2	The system shall allow users to log in and receive an authenticated session (JWT).
FR-3	The system shall allow a user to create a new organization, automatically becoming its Admin.
FR-4	The system shall allow an Admin to invite or add members to their organization (via email or invite code — the simplest viable mechanism for the MVP).
FR-5	The system shall enforce that every user action is scoped to the authenticated identity and role of that user.

6.2 Organization Mode

ID	Requirement
FR-6	The system shall allow an Organization Admin to upload documents (PDF, DOCX, TXT) to the organization's knowledge base.
FR-7	The system shall allow an Organization Admin to delete documents from the organization's knowledge base.
FR-8	The system shall allow Organization Members to ask questions against the organization's knowledge base without upload permissions.
FR-9	The system shall ensure an organization's documents and vectors are never accessible to another organization.
FR-10	The system shall ensure Members cannot access Admin-only actions (upload/delete).

6.3 Personal Mode

ID	Requirement
FR-11	The system shall allow a Personal User to upload their own documents (PDF, DOCX, TXT).
FR-12	The system shall allow a Personal User to delete their own documents.
FR-13	The system shall ensure Personal Mode documents are never accessible from any Organization context, and vice versa.

6.4 RAG / Chat Functionality

ID	Requirement
FR-14	The system shall extract text from uploaded PDF, DOCX, and TXT files.
FR-15	The system shall split extracted text into chunks suitable for embedding.
FR-16	The system shall generate vector embeddings for each chunk and store them, tagged with their owning organization or personal user.
FR-17	The system shall accept a natural language question and retrieve the most relevant chunks, scoped only to the requester's organization or personal space.
FR-18	The system shall generate an answer using only the retrieved context.
FR-19	The system shall respond with a clear "not found in the documents" message when no relevant context is retrieved.
FR-20	The system shall support switching the LLM provider (OpenAI/Anthropic) via configuration, without code changes to the retrieval logic.

6.5 Chat History

ID	Requirement
FR-21	The system shall display chat messages within an active session.
FR-22	The system should persist chat history so it survives application restarts (Should-Have).

6.6 Cross-Platform Client

ID	Requirement
FR-23	The system shall provide a Flutter-based client functioning on Web and Android.
FR-24	The system should also function on iOS and Windows if the timeline permits (Should-Have).

7. Non-Functional Requirements

ID	Category	Requirement
NFR-1	Security	Passwords must be stored hashed (never in plaintext); API keys must never be exposed to the client.
NFR-2	Security / Isolation	Data isolation between tenants (organizations, personal users) must be enforced at the query/database level, not merely hidden in the user interface.
NFR-3	Performance	Typical chat responses should return within approximately 10 seconds.
NFR-4	Usability	The UI must be usable without instructions by a non-technical user.
NFR-5	Reliability	The system must handle LLM/API failures gracefully without crashing.
NFR-6	Maintainability	Code must be modular and documented, separating RAG logic, authentication logic, and API layers.
NFR-7	Portability	The Flutter client must run from a single codebase across all supported platforms.
NFR-8	Scalability (Design-Level)	The architecture should be designed so that additional organizations or users could be added without a full redesign; actual load-testing is not required at this stage.
NFR-9	Responsiveness (UI)	The UI must adapt cleanly to different screen sizes (phone, web, desktop window).

8. User Stories

ID	User Story
US-1	As a new user, I want to register and log in, so that my data and organization access are secured under my own account.
US-2	As a user, I want to create an organization, so that my team can share a common knowledge base.
US-3	As an Organization Admin, I want to upload documents to my organization, so that members can ask questions about them.
US-4	As an Organization Admin, I want to delete outdated documents, so that the chatbot does not reference obsolete information.
US-5	As an Organization Member, I want to ask questions about company documents without needing upload access, so that I can quickly find information relevant to my role.
US-6	As a Personal User, I want to upload and query my own private documents, so that I get personal assistance without mixing my data with any organization.
US-7	As any user, I want to be told clearly when the system does not know an answer, so that I am not misled by an incorrect response.
US-8	As a user, I want to use the same app on my phone or in my browser, so that I can access my knowledge assistant from any device.
US-9	As an Organization Admin, I want assurance that other organizations cannot see my documents, so that I can trust the platform with sensitive company data.
US-10	As a developer/maintainer, I want to switch LLM providers via configuration, so that I can control cost and performance without rewriting code.

9. Use Cases

Use Case UC-1: Admin Uploads a Document

Field	Description
Actor	Organization Admin
Precondition	Admin is logged in and belongs to an organization.
Main Flow	Admin selects "Upload Document" → selects file → system validates type/size → extracts text → chunks → embeds → stores in the vector database tagged with the organization ID → confirms success to the Admin.
Alternate Flow	Invalid file type or corrupted file → system shows an error and no data is stored.

Use Case UC-2: Member Asks a Question

Field	Description
Actor	Organization Member
Precondition	Member is logged in and the organization has at least one document uploaded.
Main Flow	Member types a question → system embeds the question → retrieves top-matching chunks scoped to that organization only → builds a prompt → calls the LLM → returns a grounded answer.
Alternate Flow	No relevant chunks are found → system responds: "I couldn't find this in the organization's documents."

Use Case UC-3: Personal User Manages Own Documents

Field	Description
Actor	Personal User
Precondition	User is logged in in Personal Mode.
Main Flow	User uploads or deletes their own documents, independent of any organization; queries are scoped only to their personal document set.

Use Case UC-4: Admin Deletes a Document

Field	Description
Actor	Organization Admin
Precondition	Admin is logged in and belongs to an organization with existing documents.
Main Flow	Admin selects a document to delete → system removes the file record from the relational database and its associated vectors from ChromaDB → subsequent queries no longer retrieve that content.

Use Case UC-5: Unauthorized Access Attempt

Field	Description
Actor	Organization Member (attempting an Admin action)
Precondition	Member is logged in.
Main Flow	Member attempts to access an upload/delete endpoint → system checks the user's role → request is rejected with a clear "insufficient permissions" response.

10. Complete End-to-End Workflow

A. Organization Setup

- A user registers and creates an organization, becoming its Admin.
- The Admin invites members via email or invite code.
- The Admin uploads organizational documents.
- Each document is validated, its text extracted, chunked, embedded, and stored in ChromaDB tagged with the organization's unique ID, alongside a metadata record in the relational database.

B. Organization Querying

- A Member (or Admin) logs in and asks a question in the chat interface.
- The backend verifies the user's organization membership and scopes the search to that organization's data only.
- The question is embedded and compared against the organization's stored vectors only (never another organization's).
- Top-matching chunks are retrieved, inserted into a prompt with grounding instructions, and sent to the configured LLM.
- The answer is returned and displayed, with "not found" shown explicitly if no relevant chunks exist.

C. Personal Mode (Parallel, Isolated Path)

- A Personal User logs in and uploads their own documents, processed identically (extract, chunk, embed, store) but tagged with their personal user ID rather than any organization ID.
- Questions from this user are scoped only to their personal documents; the retrieval query never touches organizational data, and vice versa.

D. Response & History

- All answers are displayed in the chat window and appended to that session's chat history, persisted where the corresponding Should-Have requirement is implemented.

11. Edge Cases

Edge Case	Handling
Member attempts to upload/delete a document	Reject with "insufficient permissions" (role check enforced at the API level).
User's query attempts to reach another organization's data (e.g., via a manipulated request)	The backend must independently verify organization membership server-side on every request; the client-provided organization ID is never trusted alone.
Organization has zero documents uploaded	Chatbot responds: "No documents have been uploaded to this organization yet."
Personal user has zero documents uploaded	Chatbot responds: "Please upload a document first."
Admin deletes a document while a query is mid-flight	Acceptable for the MVP to complete the in-flight query using the not-yet-fully-removed data; the next query reflects the deletion.
Duplicate organization names	Uniqueness is enforced on organization identifiers (e.g., a slug); recommended over allowing duplicates with only unique internal IDs.
Corrupted or empty document upload	Detect the extraction failure and reject with a clear error; no partial data is stored.
User tries to join an organization without a valid invitation	Reject unless a valid invite code or mechanism is presented.
Extremely large document	Enforce a maximum file size (e.g., 20 MB) to keep processing time and cost predictable.
LLM API failure or timeout	Catch the failure gracefully, show a retry-able error message, and do not crash the session.

12. Assumptions

- Users have consistent internet access, which is required for LLM API calls.
- Both developers have basic familiarity with Python and general programming logic before starting; Dart/Flutter and FastAPI specifics will be learned during the project.
- A valid OpenAI or Anthropic API key with sufficient quota will be available throughout development.
- The organization invite mechanism can be a simple shareable invite code rather than a full email-invitation-with-approval workflow.
- iOS builds are feasible only if a Mac (or a cloud Mac service) is accessible; this is not guaranteed and is treated as a stretch goal accordingly.

13. Constraints

- Timeline: approximately two months in total, shared between two beginner developers.
- Team size/experience: both developers are new to AI/RAG, Flutter, and likely FastAPI; learning time must be budgeted into the schedule rather than treated as separate from development time.
- Budget: LLM API usage incurs real cost per request; testing volume should be mindful of API credit limits.
- Platform testing: iOS testing requires macOS hardware, which may not be available, constraining what can realistically be validated.
- No dedicated DevOps/infrastructure budget: deployment will use free or low-cost tiers (e.g., Render, Railway, or similar), not enterprise infrastructure.

14. Success Criteria

The project will be considered successful if, by the end of the two-month period, the following conditions are met:

- A user can register, create an organization, and be assigned as its Admin.
- An Admin can upload and delete documents, and Members can query them, all correctly scoped to that organization only.
- A Personal User can independently upload and query their own documents, fully isolated from any organization data.
- The chatbot answers strictly from retrieved document content and clearly states when it cannot find an answer.
- Data isolation between organizations, and between personal and organizational data, is verifiably correct (explicitly tested, not merely assumed).
- The Flutter client runs successfully on at least Web and Android.
- The codebase follows a clean, modular architecture with reasonable documentation and error handling.

15. Risks

Risk	Likelihood	Impact	Mitigation
Multi-tenancy isolation implemented incorrectly (data leak between organizations)	Medium	High	Dedicate explicit test cases for cross-organization access attempts; treat isolation as a first-class requirement, not an afterthought.
Scope creep back toward the full original vision mid-project	High	High	Refer back to this SRS's prioritization when tempted to add features mid-build.
Flutter/Dart learning curve slows frontend weeks	Medium	Medium	Budget extra learning time in the frontend weeks; keep the UI intentionally simple in the MVP.
Authentication/security implemented insecurely (a common beginner mistake)	Medium	High	Use established libraries (e.g., passlib/bcrypt for hashing, well-known JWT libraries) rather than custom-built cryptography.
Two developers working on tightly coupled code causes merge conflicts and delays	Medium	Medium	Split work along clear boundaries (e.g., one on backend/RAG, one on the Flutter client and integration), using Git branches per feature.
LLM API costs exceed budget during testing	Low–Medium	Medium	Use a cheaper or free-tier model during development; reserve premium calls for final testing.
iOS/Windows builds not achievable in time	Medium–High	Low	Already scoped as stretch goals rather than MVP requirements; no impact on core success criteria.

16. Future Scope

The following features are recognized as valuable extensions of the platform but are explicitly deferred beyond the current two-month development cycle:

- Fine-grained roles (Employee, Intern, Visitor) with distinct permissions.
- True anonymous/public-facing chatbot widget for external visitors.
- In-place document editing and version history.
- Third-party single sign-on (SSO), e.g., Google or Microsoft login.
- Admin analytics dashboard (usage statistics, most-asked questions, audit logs).
- OCR support for scanned/image-based PDFs.
- Multi-language support.
- iOS and Windows platform builds (if not completed as stretch goals).
- Horizontal scaling and enterprise-grade deployment infrastructure.